

ColdCast: Calibrated Cold-Start Prediction of AI System Performance

Yi Chen Kuo, Sudip Manchare, Jaideep Manupati, Akshada Shailesh Rane
Department of Computer Science, Virginia Tech, Alexandria, Virginia, USA
{alvin9840, akshada, jaideepm21, msudip}@vt.edu

Abstract—Evaluating an AI system across every item in every benchmark is prohibitively expensive, while single aggregate accuracies obscure capability and item difficulty. We propose *ColdCast*, a deep learning framework predicting the probability that an AI system answers an unseen evaluation item correctly without executing the target system. Aligned with the Stanford AIMS Predictive AI Evaluation Competition, inputs comprise system metadata, item semantics, public response logs, and a five-label adaptation budget per test system. ColdCast encodes item semantics with a pretrained text encoder, models AI systems via learned metadata embeddings, and fuses these representations for calibrated correctness estimation. We evaluate whether semantic item representations, learned system embeddings, retrieval evidence, and five-label adaptation surpass rate-based and standard neural baselines under strict item-held-out validation. Brier score serves as the primary metric alongside expected calibration error and reliability diagrams. ColdCast yields a reproducible pipeline and competition-ready artifact.

Index Terms—AI evaluation, calibration, cold-start prediction, deep learning, large language models, uncertainty estimation.

I. INTRODUCTION AND MOTIVATION

Evaluating frontier AI systems across thousands of benchmark items incurs steep compute and API costs. Furthermore, aggregate benchmark accuracy conflates a strong model failing an adversarial prompt with a weak model failing a trivial one. Predictive evaluation addresses this by estimating correctness probabilities for unobserved items without model execution.

The Stanford AIMS Predictive AI Evaluation Competition [1] formalizes this setup: models predict $\hat{p} = P(y = 1 \mid s, i)$ for subject s and item i , where $y \in \{0, 1\}$. Test items originate from held-out benchmarks, enforcing a benchmark-level cold start. Predictions must leverage public logs, metadata, semantics, and an adaptive budget of five revealed labels per test subject.

We present ColdCast, an inductive framework fusing semantic text encodings with structured metadata embeddings to yield calibrated probabilities on unseen benchmarks. Optimized for the Brier score, ColdCast balances discrimination and probabilistic reliability for efficient surrogate evaluation.

A. Research Question and Objectives

Can semantic item representations, learned AI-system representations, retrieval-based evidence, and five-label adaptation improve calibrated cold-start prediction of AI correctness beyond rate-based and standard neural baselines?

Objectives: (1) establish an item-held-out, leakage-safe validation protocol; (2) benchmark statistical and neural

baselines; (3) train a text-plus-metadata fusion model; (4) apply probability calibration; (5) explore retrieval and few-label adaptation; and (6) package a reproducible competition inference artifact.

II. RELATED WORK AND RESEARCH CONTRIBUTION

A. Related Work

Truong et al. propose amortized model evaluation via item content and adaptive testing [2], while Zhuang et al. apply computerized adaptive testing to evaluate LLMs efficiently [3]. Classical Item Response Theory (IRT) models latent ability and difficulty, but small, rapidly evolving LLM populations can violate IRT assumptions [4]. Shang et al. blend contextual item embeddings with multidimensional IRT, highlighting cross-scenario transfer as an open challenge [5]. The AIMS benchmark formalizes this challenge under cold-start conditions [1].

B. Research Contribution

ColdCast provides a controlled empirical evaluation across:

- **Item representations:** Pretrained text embeddings with structured task, tool, and type features.
- **System representations:** Learned categorical embeddings for model, provider, harness, and reasoning effort.
- **Interaction modeling:** Fusion MLP and neural factorization.
- **Retrieval evidence:** Similarity-weighted training-item statistics.
- **Calibration:** Temperature scaling evaluated on reliability diagrams.
- **Few-label adaptation:** Incorporating five revealed labels via random, uncertainty, and diversity policies.

III. DATA, BASELINES, AND EVALUATION PLAN

A. Data and Leakage-Safe Validation

Using the Stanford AIMS public release [1], each instance contains system metadata, item text, associated tools, and historical outcomes. A project data card will catalog schema distributions, missingness, and cardinality.

Random row splits leak item difficulty across partitions. Thus, our offline protocol holds out *entire evaluation items*. Preprocessing, embeddings, frequency smoothers, and retrieval indices are fit solely on training partitions. Calibration parameters are learned on a dedicated calibration split.

TABLE I
BASELINE AND COMPARISON MODELS.

Method	Description
Constant 0.5	Predict $P(\text{correct}) = 0.5$ across all pairs.
Global rate	Predict global training-set correctness rate.
Per-system rate	Smoothed historical correctness rate per system.
Metadata-only	MLP trained on structured system metadata.
Text-only	MLP on item embeddings and structured item features.
Retrieval-only	Similarity-weighted statistics from training items/systems.
Fusion MLP	Learned joint item and system representations.
Factorization	Latent system-item interactions with content features.

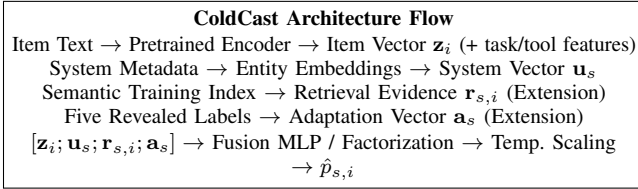


Fig. 1. ColdCast system workflow and modular extension paths.

B. Baselines and Evaluation Metrics

Table I outlines comparison baselines spanning heuristic, single-modality, and interaction models.

Our primary evaluation metric is the Brier score:

$$\text{Brier} = \frac{1}{N} \sum_{n=1}^N (\hat{p}_n - y_n)^2, \quad (1)$$

where $\hat{p}_n \in [0, 1]$ and $y_n \in \{0, 1\}$. We also report expected calibration error (ECE), binary cross-entropy, reliability diagrams, and bootstrap confidence intervals. Ablations isolate the marginal value of text, metadata, fusion architectures, retrieval, temperature scaling, and five-label acquisition strategies.

IV. PROPOSED DEEP-LEARNING APPROACH

A. System Overview

For system s and item i , ColdCast predicts:

$$\hat{p}_{s,i} = \sigma(f_\theta(\mathbf{z}_i, \mathbf{u}_s, \mathbf{r}_{s,i}, \mathbf{a}_s)), \quad (2)$$

where $\mathbf{z}_i$ is the item encoding, $\mathbf{u}_s$ is the system encoding, $\mathbf{r}_{s,i}$ denotes retrieval features, $\mathbf{a}_s$ is an optional five-label adaptation vector, f_θ is a neural head, and $\sigma(\cdot)$ is the sigmoid function.

B. Item and System Encoders

Items are encoded via frozen pretrained sentence transformers to preserve compute efficiency and isolate predictive contributions. Categorical task indicators, tool flags, and text length statistics are projected and concatenated with text vectors to produce $\mathbf{z}_i$. Systems are represented by learned embedding vectors for model ID, provider, harness, and reasoning effort, combined with normalized numeric features via an MLP to generate $\mathbf{u}_s$. Out-of-vocabulary regularization mitigates overfitting on rare systems.

TABLE II
PROJECT MILESTONES AND TIMELINE.

Period	Milestone and Core Deliverables
Sep. 17–22	Finalize proposal, data inspection, item split, tracker setup.
Sep. 23–Oct. 6	Ingestion pipeline, data card, leakage tests, rate baselines.
Oct. 7–20	Encoders and fusion MLP; Checkpoint I report and plots.
Oct. 21–Nov. 3	Factorization, retrieval indexing, and primary ablations.
Nov. 4–10	Calibration scaling, error analysis, Checkpoint II presentation.
Nov. 11–20	Five-label adaptation policies, competition packaging.
Nov. 21–Dec. 8	Visualizations, demo video, final oral presentation.
Dec. 9–11	Final IEEE-format paper submission, code repository freeze.

TABLE III
TEAM TASK ALLOCATIONS.

Member (Role)	Primary Responsibilities
Sudip (Integration Lead)	Scheduling, repository, experiment tracking, packaging.
Yi Chen (Data Lead)	Data pipeline, leakage checks, splits, rate baselines.
Jaideep (Modeling Lead)	Encoders, fusion MLP, factorization architectures.
Akshada (Adaptation Lead)	Retrieval index, active 5-label adaptation policies.
Collective Effort	Metrics, calibration analysis, demo, final write-up.

C. Fusion, Retrieval, and Calibration

The core architecture concatenates $\mathbf{z}_i$ and $\mathbf{u}_s$ into an MLP with dropout and sigmoid output, optimized using binary cross-entropy. A neural factorization head serves as an alternative to capture multiplicative latent interactions.

The retrieval module constructs a training-only semantic vector index, extracting similarity-weighted correctness rates from neighboring items and peer models. Uncalibrated outputs are adjusted via temperature scaling fit on a validation partition. For few-label adaptation, the five revealed labels form an adaptation vector $\mathbf{a}_s$; active label selection contrasts random, uncertainty, and diversity sampling. ColdCast is implemented in PyTorch, leveraging Virginia Tech computing resources.

V. PROJECT PLAN, TEAM ROLES, AND DELIVERABLES

A. Schedule and Team Responsibilities

Tables II and III detail the milestone schedule and functional member responsibilities across all project phases.

B. Deliverables and Scope Management

Deliverables include the reproducible GitHub codebase, dataset cards, validation scripts, trained model checkpoints, reliability figures, project webpage, demonstration video, and final report. If submission requirements are satisfied, packaged `model.py` and `labeling.py` modules will be submitted.

Required core: item-held-out validation, statistical baselines, single-modality models, fusion MLP, Brier optimization, and temperature calibration. Factorization and retrieval indices form primary extensions, while active five-label adaptation constitutes a stretch target.

VI. CONCLUSION

ColdCast provides a principled deep learning framework for predicting AI system performance under benchmark cold-start constraints. By integrating item semantics, system metadata, temperature calibration, and retrieval extensions, the project offers a computationally efficient surrogate for comprehensive LLM benchmarking.

REFERENCES

- [1] Stanford AI Measurement Science, “Predictive AI Evaluation Competition,” 2026. [Online]. Available: <https://aimslab.stanford.edu/competition/>
- [2] C. Truong et al., “Reliable and Efficient Amortized Model-based Evaluation,” in *Proc. ICML*, 2025.
- [3] J. Zhuang et al., “Efficiently Measuring the Cognitive Ability of LLMs: An Adaptive Testing Perspective,” arXiv:2306.10512, 2023.
- [4] Y. Jiang et al., “Can We Trust Item Response Theory for AI Evaluation?,” arXiv:2607.15190, 2026.
- [5] Y. Shang, Z. Tang, and X. He, “LLM Evaluation on Unseen Questions,” arXiv:2608.22295, 2026.

APPENDIX

A. Optimization and Calibration Formulations

Parameters θ are trained using binary cross-entropy with L_2 decay:

$$\mathcal{L}_{\text{BCE}}(\theta) = -\frac{1}{N} \sum_{n=1}^N [y_n \log \hat{p}_n + (1 - y_n) \log(1 - \hat{p}_n)] + \lambda \|\theta\|_2^2. \quad (3)$$

Post-hoc temperature scaling optimizes temperature $T > 0$ and bias b over calibration split $\mathcal{D}_{\text{cal}}$:

$$\min_{T, b} - \sum_{(s, i, y) \in \mathcal{D}_{\text{cal}}} \left[y \log \sigma \left(\frac{z_{s, i}}{T} + b \right) + (1 - y) \log \left(1 - \sigma \left(\frac{z_{s, i}}{T} + b \right) \right) \right]. \quad (4)$$

B. Competition Runtime Specification

ColdCast respects all official container constraints:

- **Submission Format:** Flat ZIP archive holding `model.py` (`predict(input, labeled)`), optional `labeling.py` (`acquisition_function(input)`), and `models.txt`.
- **Interface:** `input` is a 2-tuple `[subject, item]` of string key-value dictionaries. `labeled` contains ≤ 5 tuples `[[subject, item], label]`.
- **Output Criteria:** `predict()` returns a float in $[0.0, 1.0]$. NaNs or out-of-range outputs trigger failure exit code 42.
- **Offline Execution:** Strictly no runtime network access; weights specified in `models.txt` are cached locally during image pre-build.